

Student name: Gordon Tse

Student ID: s223532749

SIT225: Data Capture Technologies

Activity 2.1: Working with sensor - DHT22

DHT22 is a temperature and humidity sensor.

Hardware Required

Arduino Board

DHT22 sensor

USB cable

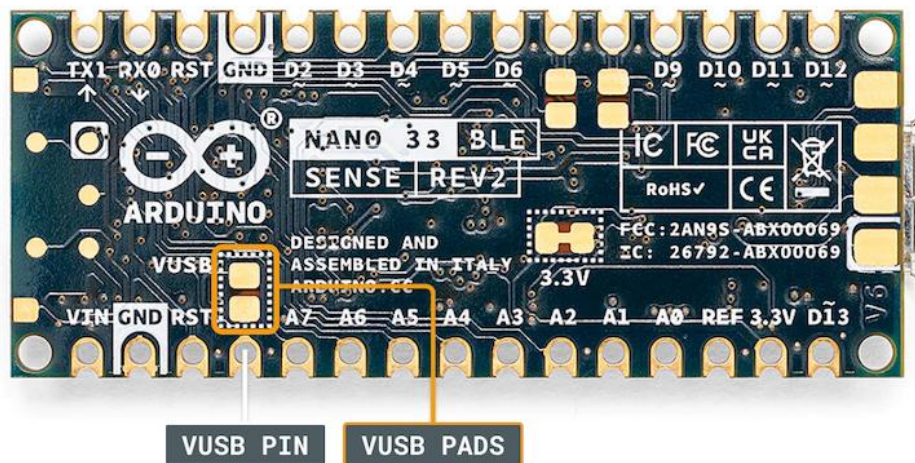
Software Required

Arduino programming environment

Known issue, action required

Arduino Nano 33 IoT board operates on 3.3 V, it needs to be arranged to make it 5 V. The Arduino board has a pin called VUSB or VBUS and there are 2 pads next to the pin. **These two pads must be shorted to enable the pin** (see detail here

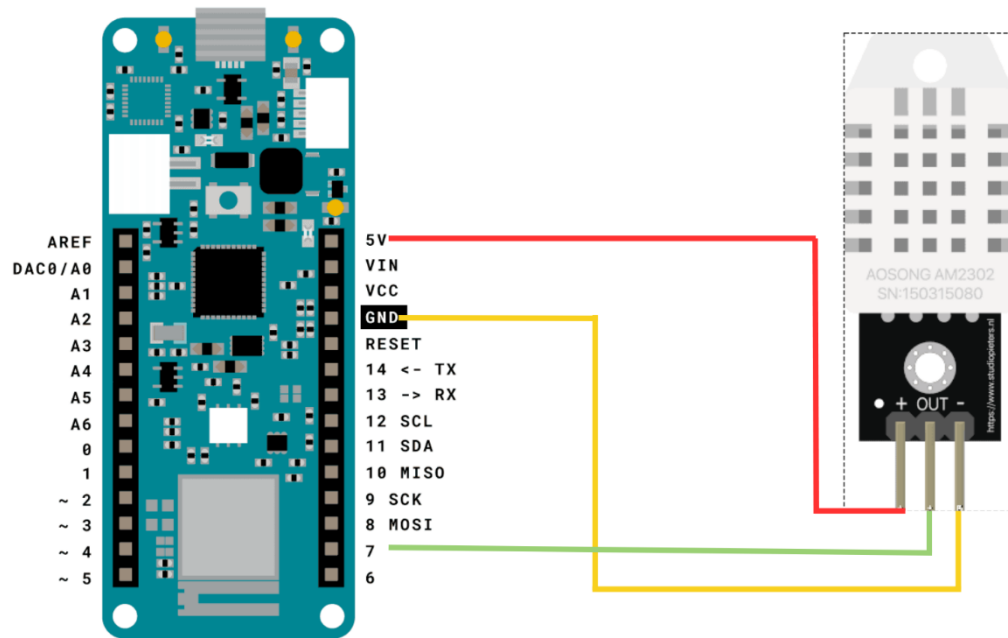
<https://support.arduino.cc/hc/en-us/articles/360014779679-Enable-5-V-power-on-the-VUSB-or-VBUS-pin-on-Nano-boards>).



To test, you can use a wire and connect these 2 pads manually by hand to see if data is coming through DHT22 sensor. For long data collection, you will need to solder the wire permanently to connect the pads. Seeking help from tutors there is an on-campus facility called Maker Space where you can do it.

Steps:

Step	Action
1	Connect your DHT22 Temperature and Humidity Sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have. The diagram illustrates two different pin configurations for a DHT22 sensor. On the left, a 4-pin header is shown with Vcc (1) in red, Data (2) in green, and Gnd (4) in black. A red 'X' is placed over the Data pin. On the right, a 3-pin header is shown with Vcc (1) in red, Data (2) in green, and Ground (3) in black.



- Pick a red male-female jumper wire and attach the female end to pin 1 (VCC pin) on the sensor. Plug the male end into the Arduino board's 5V power pin.
- Pick a blue male-female jumper wire and attach the female end to pin 2 (DATA pin) on the sensor. Plug the male end into the Arduino board's digital data pin 2.
- Pick a black male-female jumper wire and attach the female end to pin 4 (GND) on the sensor. Plug the male end into the Arduino board's GND pin.
- Sensor's pin 3 is not used.

2 Connect your Arduino board to your computer using the USB cable.

3 Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_dht22.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

	<pre> 6 7 // Add "DHT sensor library" 8 #include <DHT.h> 9 10 #define DHTPIN 2 // digital pin number 11 #define DHTTYPE DHT22 // DHT type 11 or 22 12 DHT dht(DHTPIN, DHTTYPE); 13 14 // variables to store data. 15 float hum, temp; 16 17 void setup() { 18 // Set baud rate for serial communication 19 Serial.begin(9600); 20 21 // initialise DHT library 22 dht.begin(); 23 } 24 25 void loop() { 26 // read data 27 hum = dht.readHumidity(); 28 temp = dht.readTemperature(); 29 30 // Print data to serial port - a long way 31 // 32 // Serial.print("Humid: "); 33 // Serial.print(hum); 34 // Serial.print(" %, Temp: "); 35 // Serial.print(temp); 36 // Serial.println(" Celsius"); 37 38 // Print data to serial port - a compact way 39 Serial.println(String(hum) + "," + String(temp)); 40 41 // wait a while 42 delay(15*1000); 43 } </pre>	
4	Question: A spec of the DHT22 sensor is given in the link below. It mentions that the sampling rate is 0.5 Hz. https://lastminuteengineers.com/dht11-dht22-arduino-tutorial - What does the sampling rate mean? - Where is this used in the Arduino code? Answer: Sample rate mean how many times per sec the device measure real-world signal. In the line 42 “delay (15*1000)”.	
5	Question: Take a screenshot of your Serial Monitor displaying temperature & humidity sensor data logs. Add the image here. Answer:	

→ ↻

📶 Arduino NANO 33 IoT ▾

code.ino

```
9   #include <DHT.h>
10
11   #define DHTPIN 21 // digital pin number
12   #define DHTTYPE DHT22 // DHT type 11 or 22
13   DHT dht(DHTPIN, DHTTYPE);
14
15   // variables to store data.
16   float hum, temp;
17
18   void setup() {
19       // Set baud rate for serial communication
20       Serial.begin(9600);
21
22       // initialise DHT library
23       dht.begin();
24   }
25
26   void loop() {
27       // read data
28       hum = dht.readHumidity();
29       temp = dht.readTemperature();
30
31       // Print data to serial port - a long way
```

Output Serial Monitor ✕

Message (Enter to send message to 'Arduino NANO 33 IoT' on 'COM3')

62.50,16.00
62.80,17.90
63.20,17.90
63.60,17.90
63.80,17.90
64.10,17.80
64.00,17.80
63.20,17.80
62.90,17.80
63.00,17.80
62.80,17.80
62.50,17.80

Activity 2.2: Working with sensor - HC-SR04

HC-SR04 is an Ultrasonic sensor.

Hardware Required

Arduino Board

HC-SR04 Ultrasonic sensor

USB cable

Software Required

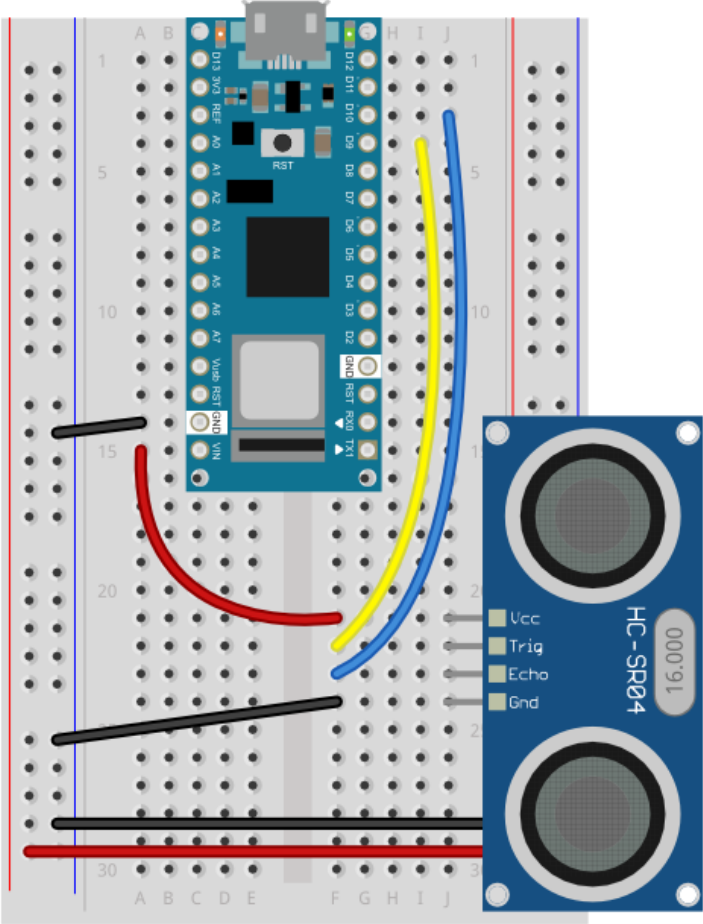
Arduino programming environment

Known issue, action required

The same known issue applies to SR04 which operates at 5 V source while the Arduino Nano 33 IoT supplies 3.3 V. It requires 2 VUSB (or VBUS) pads to be shorted to enable the pin.

Steps:

Step	Action
1	Connect your HC-SR04 sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.

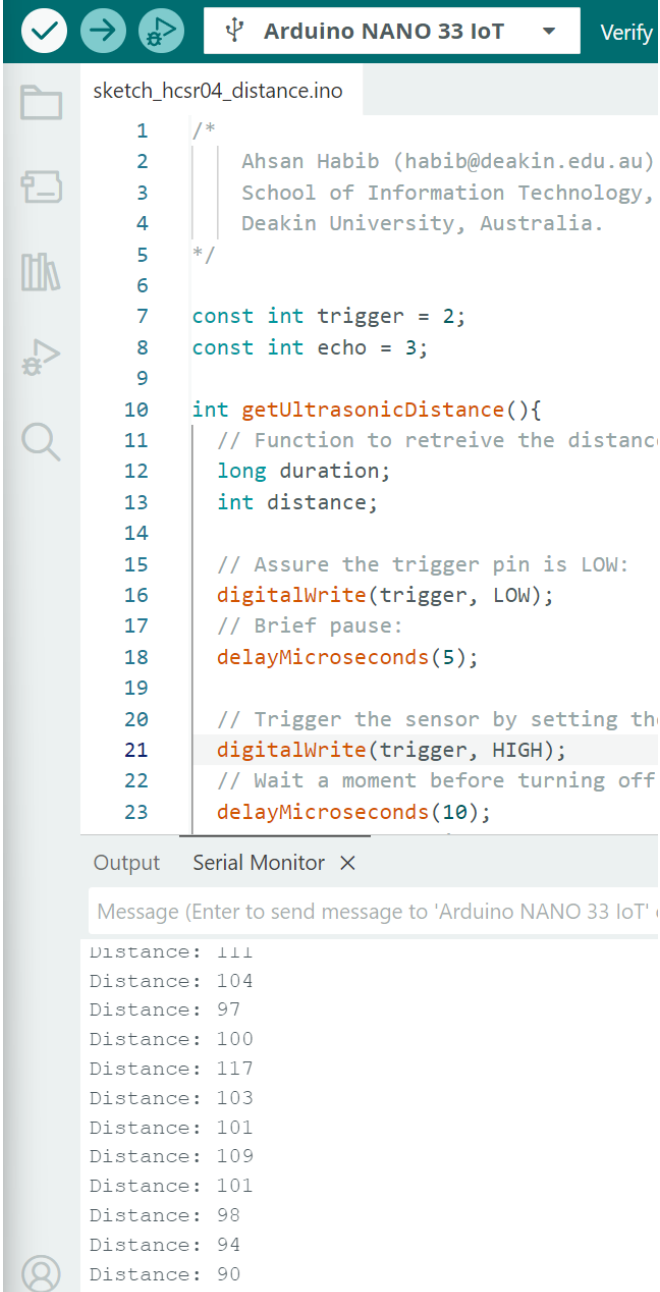
	
2	Connect your Arduino board to your computer using the USB cable.
3	Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_hcsr04_distance.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

	<pre> 7 const int trigger = 2; 8 const int echo = 3; 9 10 int getUltrasonicDistance(){ 11 // Function to retrieve the distance reading of the ultrasonic 12 long duration; 13 int distance; 14 15 // Assure the trigger pin is LOW: 16 digitalWrite(trigger, LOW); 17 // Brief pause: 18 delayMicroseconds(5); 19 20 // Trigger the sensor by setting the trigger to HIGH: 21 digitalWrite(trigger, HIGH); 22 // Wait a moment before turning off the trigger: 23 delayMicroseconds(10); 24 // Turn off the trigger: 25 digitalWrite(trigger, LOW); 26 27 // Read the echo pin: 28 duration = pulseIn(echo, HIGH); 29 // Calculate the distance in centimeter (CM): 30 distance = duration * 0.034 / 2; 31 32 // Uncomment this line to return value in IN instead of CM: 33 //distance = distance * 0.3937008 34 35 // Return the distance read from the sensor: 36 return distance; 37 } 38 39 void setup() { 40 // Define inputs and outputs: 41 pinMode(trigger, OUTPUT); 42 pinMode(echo, INPUT); 43 44 // Start the serial monitor: 45 Serial.begin(9600); 46 } 47 48 void loop() { 49 // Print the distance to the serial monitor: 50 Serial.print("Distance: "); 51 Serial.println(getUltrasonicDistance()); 52 53 // Wait one second before continuing: 54 delay(1000); 55 } </pre>
4	Question: Spec of SR04 is available here (https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf). Identify 2 critical aspects you should be careful about this sensor operation. Answer: 1. This sensor might not work with surfaces that will absorb the sound wave, also might not work with mirror so need to be careful what object we want to detect. 2. if object getting to close it might not accurate. 3. and it need time to process to get back result.

5

Question: Take a screenshot of your Serial Monitor displaying distance values while you try to generate a periodic motion by moving your hand gradually back and forth towards the sensor. Add the image here.

Answer:



The screenshot shows the Arduino IDE interface. The top bar indicates the board is 'Arduino NANO 33 IoT'. The sketch file is 'sketch_hcsr04_distance.ino'. The code is as follows:

```
1  /*
2     Ahsan Habib (habib@deakin.edu.au)
3     School of Information Technology,
4     Deakin University, Australia.
5  */
6
7  const int trigger = 2;
8  const int echo = 3;
9
10 int getUltrasonicDistance(){
11     // Function to retrieve the distance
12     long duration;
13     int distance;
14
15     // Assure the trigger pin is LOW:
16     digitalWrite(trigger, LOW);
17     // Brief pause:
18     delayMicroseconds(5);
19
20     // Trigger the sensor by setting the
21     digitalWrite(trigger, HIGH);
22     // Wait a moment before turning off
23     delayMicroseconds(10);
```

The Serial Monitor is open, showing the following output:

```
Distance: 111
Distance: 104
Distance: 97
Distance: 100
Distance: 117
Distance: 103
Distance: 101
Distance: 109
Distance: 101
Distance: 98
Distance: 94
Distance: 90
```

Activity 2.3: Working with sensor - Accelerometer

LSM6DS3 module on the Arduino Nano 33 IoT is an accelerometer and gyroscope sensor.

Hardware Required

Arduino Nano 33 IoT Board (has inbuilt LSM6DS3 module)

USB cable

Software Required

Arduino programming environment

Steps:

Step	Action
1	Write Arduino sketch (or download code from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_accelero.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

	<pre> 6 7 // Add Arduino_LSM6DS3 library 8 // from Arduino IDE Library Manager. 9 #include <Arduino_LSM6DS3.h> 10 11 float x, y, z; 12 13 void setup() { 14 Serial.begin(9600); // set baud rate 15 while (!Serial); // wait for port to init 16 Serial.println("Started"); 17 18 if (!IMU.begin()) { 19 Serial.println("Failed to initialize IMU!"); 20 while (1); 21 } 22 23 Serial.println(24 "Accelerometer sample rate = " 25 + String(IMU.accelerationSampleRate()) + " Hz"); 26 } 27 28 void loop() { 29 // read accelero data 30 if (IMU.accelerationAvailable()) { 31 IMU.readAcceleration(x, y, z); 32 } 33 34 Serial.println(35 String(x) + ", " + String(y) + ", " + String(z)); 36 37 delay(1000); 38 } </pre>
2	Question: Spec of LSM6DS3 is available here (https://content.arduino.cc/assets/st_imu_lsm6ds3_datasheet.pdf). Identify at least 3 attributes of this sensor you think important to work with. Answer: 1. Unlike simpler sensors that require the Arduino to constantly ask for data, the LSM6DS3 has a built-in 8 Kilobyte FIFO memory buffer. So it more efficient. 2. it can do sensitivity tuning which it can adjust to suitable sensitivity for different project. 3. it have both accelerometer and gyroscope
3	Question: Take a screenshot of your Serial Monitor displaying sensor readings. Add the image here. Answer:

	sketch_accelero.ino <pre> 1 /* 2 Ahsan Habib (habib@deakin.edu.au), 3 School of Information Technology, 4 Deakin University, Australia. 5 */ 6 7 // Add Arduino_LSM6DS3 library 8 // from Arduino IDE Library Manager. 9 #include <Arduino_LSM6DS3.h> 10 11 float x, y, z; 12 13 void setup() { 14 Serial.begin(9600); // set baud rate 15 while (!Serial); // wait for port to init 16 Serial.println("Started"); 17 18 if (!IMU.begin()) { 19 Serial.println("Failed to initialize IMU!"); 20 while (1); 21 } 22 23 Serial.println(</pre> Output Serial Monitor X Message (Enter to send message to 'Arduino NANO 33 IoT' on 'COM3') <pre> 0.46, -0.41, 0.18 0.44, -0.52, 0.78 0.03, -0.87, 0.59 -0.12, -0.77, 0.62 -0.12, -0.81, 0.62 -0.16, -0.62, 0.77 -0.06, 0.66, 0.78 -0.10, 0.71, 0.66 -0.25, 0.67, 0.66 -0.33, 0.65, 0.78 -0.23, 0.52, 0.90 0.07, 0.00, 1.00 </pre>
4	Question: Identify the max sampling rate and consider reducing the delay (line 37 in the sketch) to increase the number of samples. Summarise your findings here. Answer: Accelerometer sample rate = 104.00 Hz in code it showing 104 HZ so 104 times per second is the fastest sample rate. For now delay(1000) is printing out 1 per sec. by removing this line, it will output as fast as it can that is 104 per sec.

Activity 2.4: Plot data using Python Notebook

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. You can find detail in official website (<https://matplotlib.org>).

Hardware Required

No hardware required

Software Required

Python Jupyter notebook

Steps:

Step	Action
1	Download the Jupyter notebook week2_notebook.ipynb from here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/week2_notebook.ipynb). Follow the instructions in the notebook to carry out instructions and finally convert the notebook to PDF so you can combine it with this activity sheet PDF.

```
student_name = "Gordon" # fill your name
student_id = "s223532749" # fill your student ID
print("Student name: " + student_name)
print("Student ID: " + student_id)
```

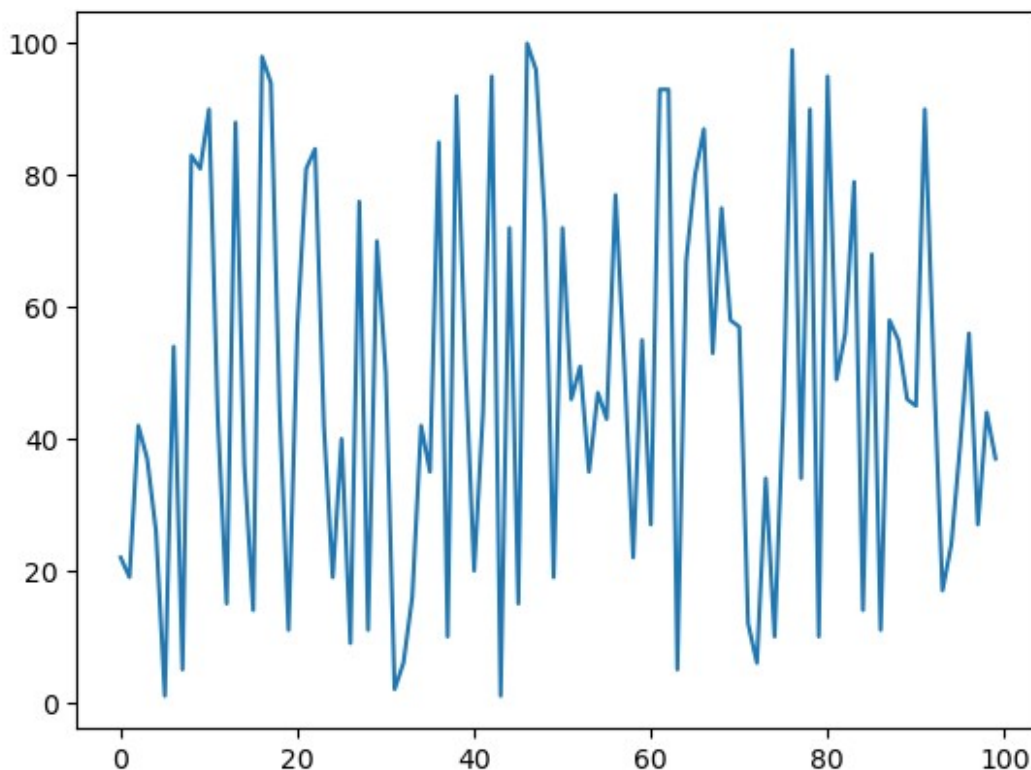
Student name: Gordon
Student ID: s223532749

```
import random
import matplotlib.pyplot as plt
```

```
n_values = 100
y_values = []
```

```
# Create data (y_values) randomly between 1 and 100.
for i in range(n_values):
    y_values.append(random.randint(1, 100))
```

```
x_values = range(n_values) # X is sequence of values 0-99
plt.plot(x_values, y_values)
plt.show()
```



```
# Plot 2 variables
#
```

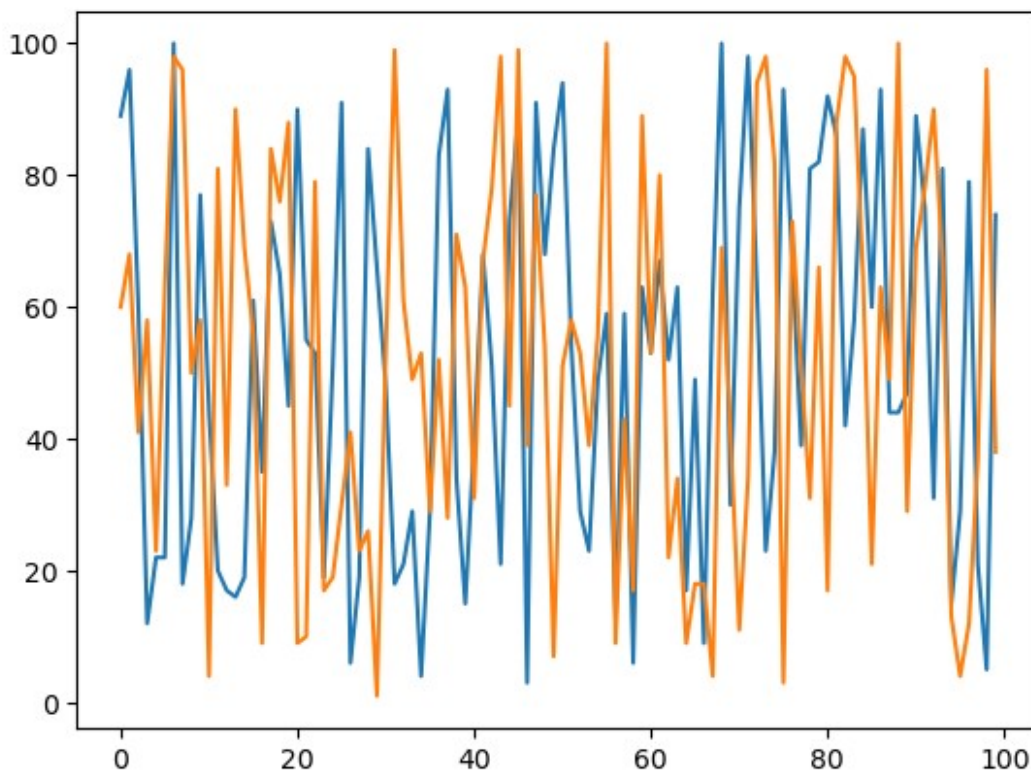
```

n_values = 100
y_values_1 = []
y_values_2 = []

# Create data (y_values) randomly between 1 and 100.
for i in range(n_values):
    y_values_1.append(random.randint(1, 100))
    y_values_2.append(random.randint(1, 100))

x_values = range(n_values) # X is sequence of values 0-99
plt.plot(x_values, y_values_1)
plt.plot(x_values, y_values_2) # call plot again draws in the same
graph.
plt.show()

```

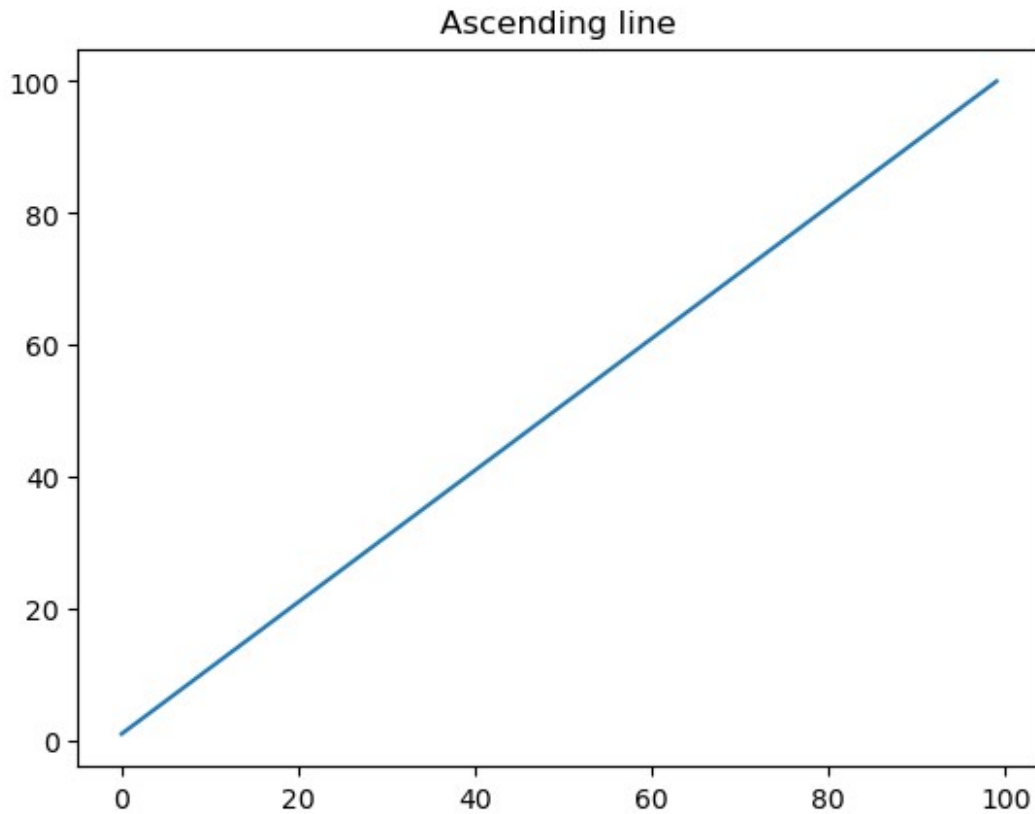


```

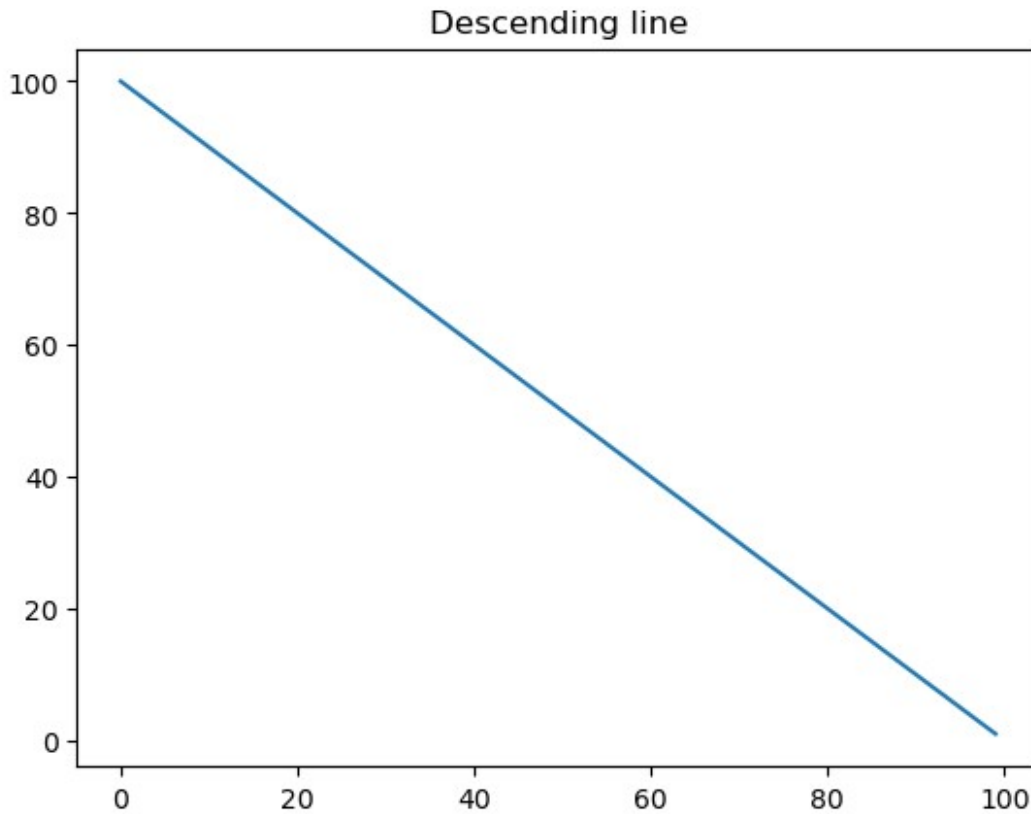
#
# Activity 1: Create data so that the plot draws an
# ascending line (y_values increase at any rate).
#

y_values_asc = list(range(1, n_values + 1))
plt.plot(x_values, y_values_asc)
plt.title("Ascending line")
plt.show()

```



```
#  
# Activity 2: Create data so that the plot draws a  
# descending line (y_values decrease at any rate).  
#  
y_values_desc = list(range(n_values, 0, -1))  
plt.plot(x_values, y_values_desc)  
plt.title("Descending line")  
plt.show()
```



```
#  
# Activity 3: Create data so that the plot draws a  
# wave. You can consider using Python's math library, which has  
# a sin function (detail  
https://www.w3schools.com/python/ref\_math\_sin.asp).  
#  
import math  
  
amplitude = 50  
cycles = 3 # number of full sine cycles across the x range  
y_wave = [amplitude * math.sin(2 * math.pi * cycles * (i / n_values))  
for i in x_values]  
  
plt.plot(x_values, y_wave)  
plt.title("Sine wave")  
plt.show()
```

Sine wave

